

06 - FreeRTOS Tasks, Queues & Semaphores - Session Notes

FreeRTOS Tasks, Queues and Semaphores

1. What Is FreeRTOS and Why Does It Exist?

When you write firmware for a real IoT product it needs to do many things at the same time:

```
Read sensor every 2 seconds
Publish to MQTT every 5 seconds
Watch for button press anytime
Blink status LED every 500ms
Handle WiFi reconnection if dropped
Monitor battery level every 600 seconds
```

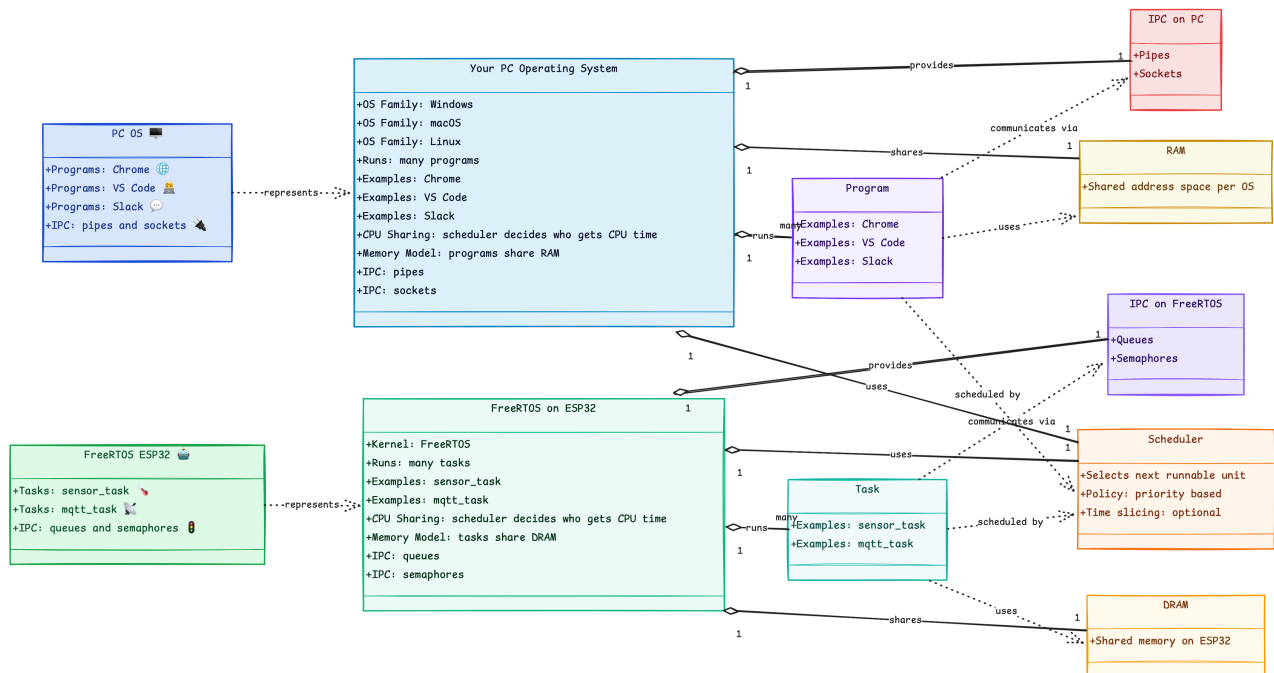
Without an operating system — you would write one giant loop trying to juggle all of this manually. It becomes unmaintainable, unreliable and nearly impossible to debug.

FreeRTOS solves this. It is a Real Time Operating System that runs on your ESP32 and gives you:

- Tasks — independent units of work running concurrently
- Queues — safe data transfer between tasks
- Semaphores — safe sharing of resources between tasks
- Timers — precise periodic execution
- Watchdogs — detecting when tasks get stuck

PC Analogy

YOUR PC OPERATING SYSTEM	FREERTOS ON ESP32
Windows / macOS / Linux	FreeRTOS
Runs many programs	Runs many tasks
Chrome, VS Code, Slack	sensor_task, mqtt_task
Scheduler decides who	Scheduler decides who
gets CPU time	gets CPU time
Programs share RAM	Tasks share DRAM
IPC — pipes, sockets	Queues, semaphores



Key difference: FreeRTOS is deterministic. When a high priority task needs to run — it runs immediately. Windows is not — it might delay your task by milliseconds unpredictably. For IoT products, determinism matters.

2. Tasks — The Foundation

A task in FreeRTOS is like a program running on your PC. Each task has its own:

- Stack (private memory for local variables)
- Priority (determines who runs when)
- State (running, ready, blocked, suspended)

Creating a Task

```
void my_task(void *arg)
{
    while (1) {
        ESP_LOGI("task", "I am running");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
    /* Tasks should never return.
     * If they do — call vTaskDelete(NULL) */
}

void app_main(void)
{
    xTaskCreate(
        my_task,          /* Function to run as task */
        "my_task",        /* Name — for debugging only */
        2048,             /* Stack size in bytes */
        NULL,             /* Parameter passed to task */
```

```

    5,          /* Priority – higher = more important */
    NULL       /* Handle – NULL if not needed */
);
}

```

Task Parameters Explained

Parameter	What it means	Typical value
Function	The function that IS the task	your function
Name	String label for debugger	"sensor_task"
Stack size	Private RAM in bytes	2048 to 8192
Parameter	Pointer passed to function	NULL or &config
Priority	0 = lowest, 25 = highest	5 for most tasks
Handle	Pointer to control task later	NULL if not needed

Priority Rules

```

Priority 25  ← Highest – runs before everything
Priority 10  ← High
Priority  5  ← Medium – most application tasks
Priority  1  ← Low
Priority  0  ← Idle task – only runs when nothing else needs CPU

```

ESP32 Reserved Priorities (do not use these):

```

Priority 24  ← WiFi task
Priority 22  ← Bluetooth task
Priority 23  ← System event task

```

Your tasks should stay below 20.

Task States

The Setup

Imagine a team of employees (tasks) working in an office. There is only ONE computer (CPU) in the office that everyone shares.

RUNNING — "At the Computer"

```

Only ONE employee can sit at the computer at a time.
This is the task currently using the CPU.
Everyone else is waiting for their turn.

```

READY — "Standing in Queue at the Computer"

The employee has everything they need to work.
Their document is ready. Their hands are ready.
They are just waiting for the person at the computer to finish and give them their turn.

Multiple employees can be READY at the same time.
The most senior person (highest priority) gets to go next.

BLOCKED — "Waiting for Something Before They Can Work"

Two types of waiting:

Type 1 – Waiting for time:

"I sent an email. Now I wait 30 minutes for a reply before I do anything."
-> `vTaskDelay()` – task is sleeping

Type 2 – Waiting for a resource:

"I need the printer but someone else is using it. I cannot work until it is free."
-> `xQueueReceive()` or `xSemaphoreTake()`
task is waiting for data or a resource

Blocked employees are NOT in the queue for the computer. They are sitting at their desk doing nothing. CPU is free for other employees while they wait.

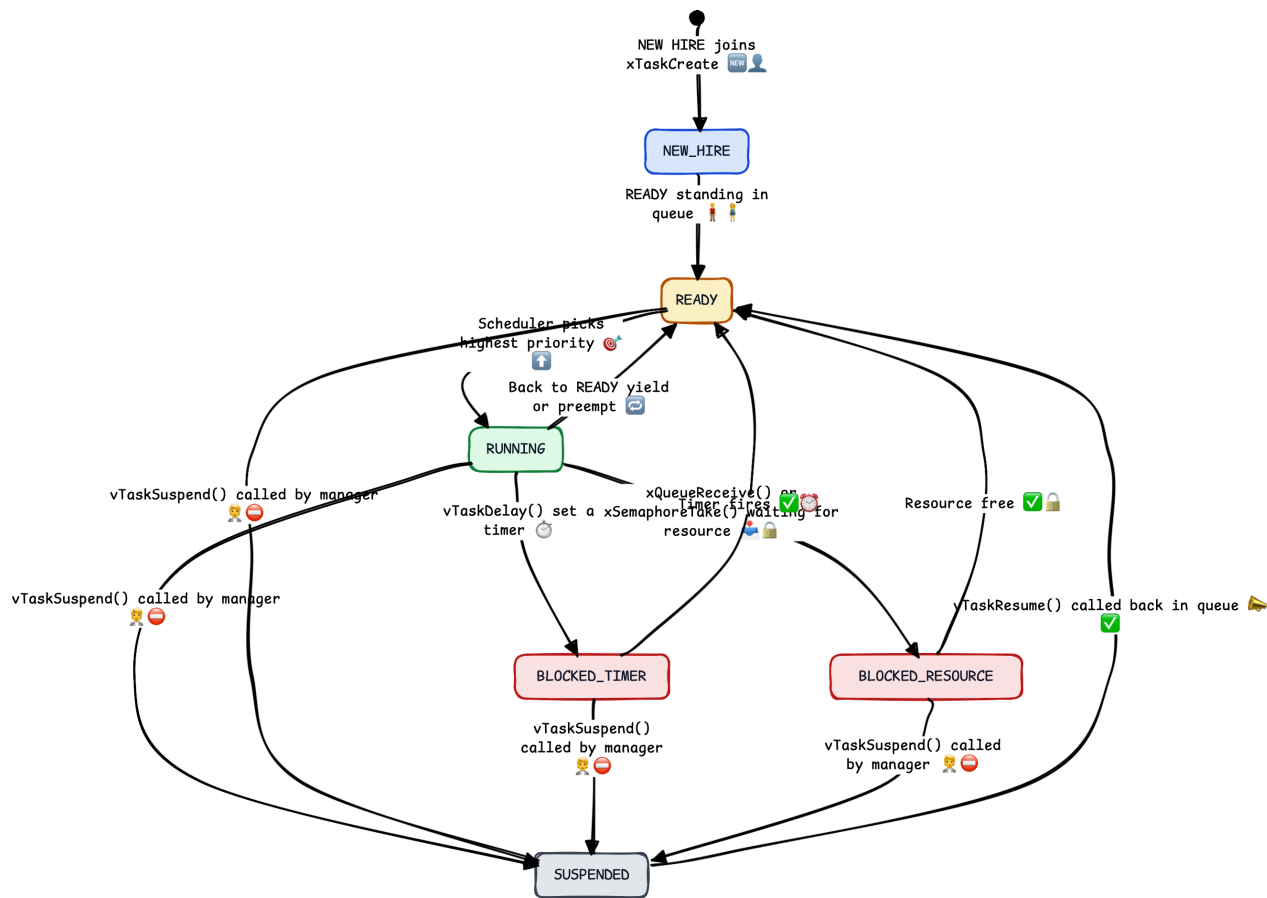
SUSPENDED — "Called in Sick — Completely Offline"

Manager explicitly told them to stop working.
They will not come back until the manager personally calls them back.

In code:

`vTaskSuspend(handle)` -> send home
`vTaskResume(handle)` -> call back

The Full Picture

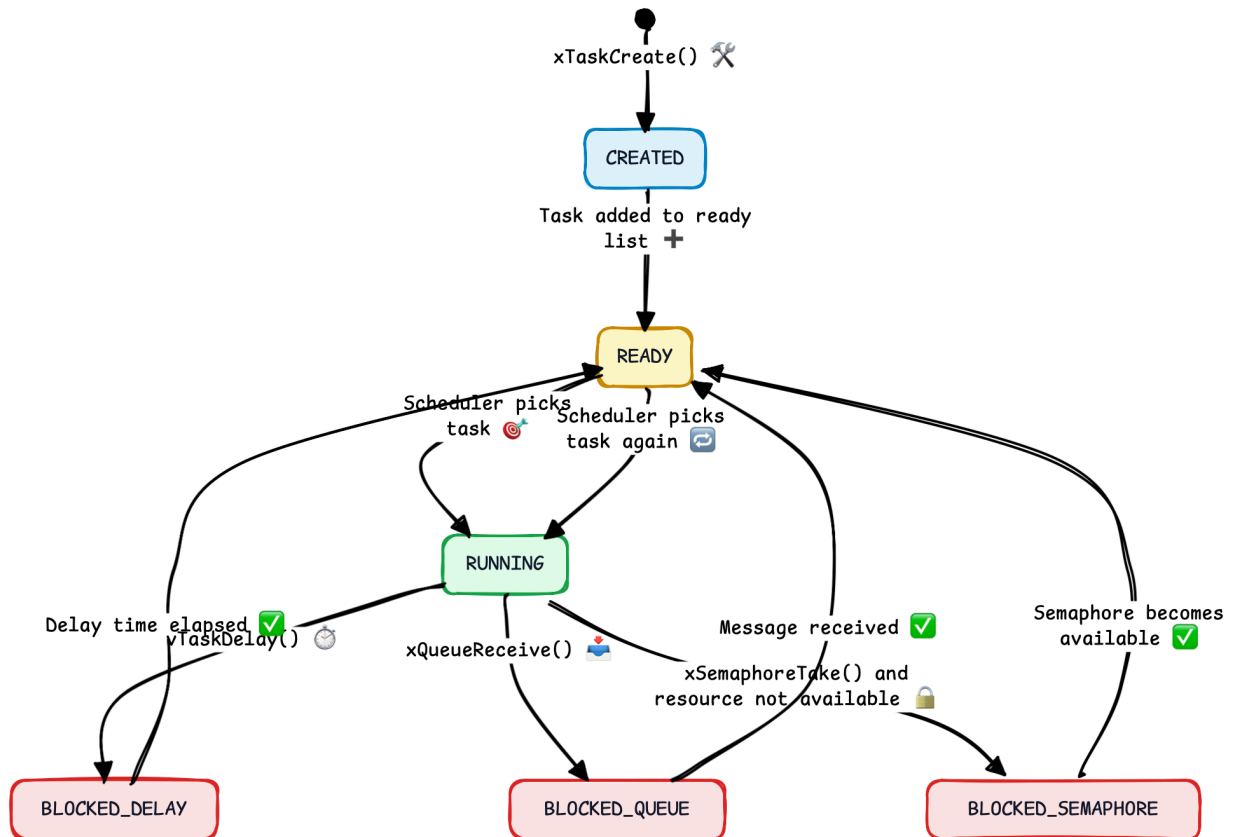


One Line Summary

State	What it means	Real world
RUNNING	Using the CPU right now	At the computer
READY	Has work to do, waiting for CPU	Standing in queue
BLOCKED	Waiting for time or a resource	Sitting at desk waiting
SUSPENDED	Explicitly stopped by manager	Called in sick

One Line to Say in Class

"RUNNING means at the computer. READY means standing in line. BLOCKED means sitting at your desk waiting for something. SUSPENDED means sent home by the manager."



vTaskDelay vs delay()

This is the most important concept in FreeRTOS:

```

/* Arduino delay() – WRONG in production */
delay(1000);
/* CPU does NOTHING for 1 second.
   No other tasks can run.
   WiFi packets missed.
   MQTT drops.
   Button presses missed. */

/* FreeRTOS vTaskDelay – CORRECT */
vTaskDelay(pdMS_TO_TICKS(1000));
/* This task sleeps for 1 second.
   CPU is RELEASED to other tasks.
   WiFi keeps running.
   MQTT keeps running.
   Other tasks execute normally. */

```

PC Analogy for vTaskDelay

`vTaskDelay` is like pressing Minimize on a window. Your program is still open – just not using the screen. Other programs get to use the computer. After 1 second Windows brings your window back.

`delay()` is like one program FREEZING your entire PC.


```
void task_b(void *arg) {
    while (1) {
        ESP_LOGI("B", "Task B running");
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

Output (alternates fairly):

```
A: Task A running
B: Task B running
A: Task A running
B: Task B running
```

FreeRTOS round-robins between equal priority tasks. Each gets one tick (1ms) before switching.

Priority Inversion Danger

FreeRTOS Priority Inversion — The Intern Blocking the CEO

The Characters

```
CEO      = HIGH priority task   (priority 10)
Manager  = MEDIUM priority task (priority  5)
Intern   = LOW priority task    (priority  1)
```

```
The Meeting Room = Shared resource (mutex)
```

Normal Day — No Problem

```
CEO needs the meeting room.
Meeting room is free.
CEO walks in, does his work, walks out.
Everyone is happy.
```

The Problem Day — Priority Inversion

```
Step 1:
Intern books the meeting room (takes the mutex).
Intern is doing some low priority paperwork inside.

Step 2:
CEO urgently needs the meeting room (tries to take mutex).
Meeting room is occupied by the Intern.
CEO CANNOT enter. CEO WAITS outside.
```


Step 3:

Manager walks by. Manager has no interest in the meeting room. Manager just keeps doing his work. Manager is priority 5 – higher than Intern priority 1. So Manager keeps running – Intern cannot finish.

Step 4:

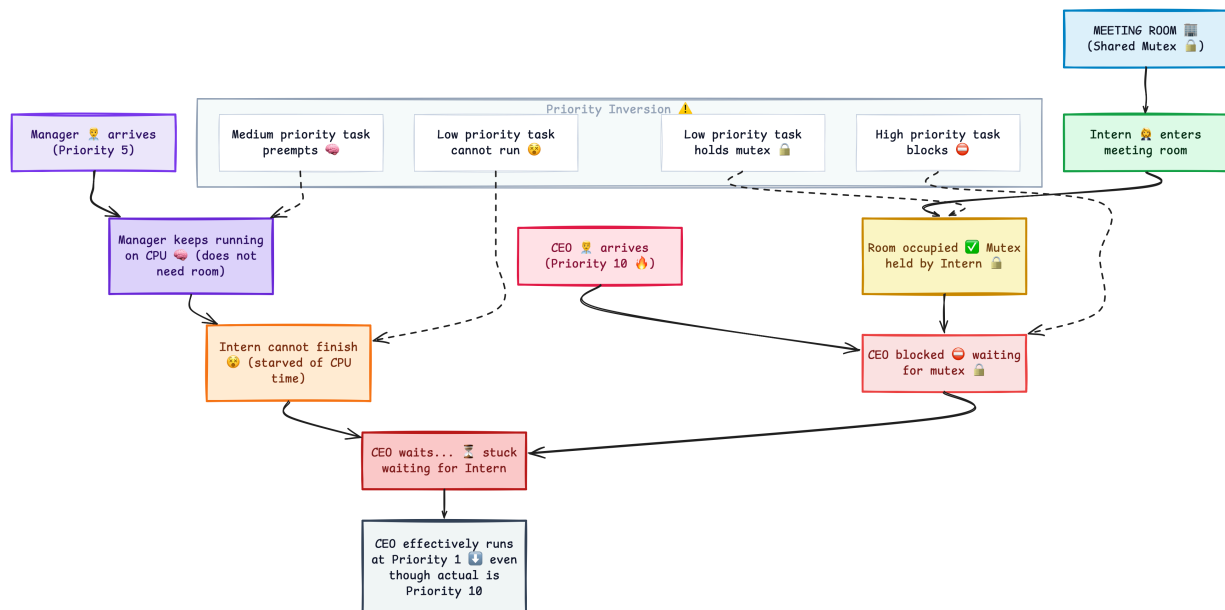
Intern is stuck – cannot finish because Manager keeps getting CPU time instead.
CEO is stuck – waiting for Intern to finish.
Manager is running freely – does not even need the room.

Result:

CEO (priority 10) is effectively blocked by Manager (priority 5) – even though Manager does not hold the resource CEO needs.

CEO is running at the LOWEST priority in the system.
This is Priority Inversion.

The Full Picture



Why This Is Dangerous in Real Products

CEO = WiFi reconnection task (must run within 5 seconds)
Manager = Data logging task (not urgent at all)
Intern = Sensor reading task (holds I2C mutex)

```
WiFi reconnection task waits for I2C mutex.  
Intern sensor task cannot finish because logger keeps running.  
WiFi reconnection times out.  
Device disconnects from network.  
Customer support gets a call.
```

Real World Example — Mars Pathfinder 1997

This is exactly what happened on NASA's Mars Pathfinder mission.

<https://kwahome.medium.com/the-first-bug-on-mars-os-scheduling-priority-inversion-and-the-mars-pathfinder-53586a631525>

```
The spacecraft kept resetting itself on Mars.  
Engineers could not understand why.
```

Root cause:

```
A low priority meteorological task held a shared mutex.  
A high priority information bus task needed the same mutex.  
A medium priority communications task kept running.
```

```
The high priority task was effectively blocked by  
the medium priority task.
```

```
System watchdog detected the high priority task was  
not running and reset the entire spacecraft.
```

```
Fix: Enable priority inheritance on the mutex.  
    One config change. Problem solved.  
    Deployed as a software patch to Mars.
```

| This bug became the most famous priority inversion in software engineering history.

The Fix — Priority Inheritance

FreeRTOS mutexes fix this automatically with Priority Inheritance.

```
When CEO is waiting outside the meeting room:
```

```
FreeRTOS sees:
```

```
"CEO (priority 10) is waiting for Intern (priority 1)  
to release the mutex."
```

```
FreeRTOS temporarily promotes Intern to priority 10.
```

```
Now:
```

```
Intern runs at priority 10 — same as CEO.  
Manager (priority 5) cannot interrupt Intern anymore.  
Intern finishes paperwork quickly.
```

Intern leaves the room – drops back to priority 1.
CEO enters the room immediately.
Everything returns to normal.

Mutex vs Binary Semaphore — The Key Difference

Feature	Mutex	Binary Semaphore
Priority inheritance	YES — automatic	NO
Use for shared resources	YES	NO
Use for signalling events	NO	YES
One line to remember	Protects resources	Signals events

```
/* Protects shared resource – use mutex */  
SemaphoreHandle_t i2c_mutex = xSemaphoreCreateMutex();  
  
/* Signals an event – use binary semaphore */  
SemaphoreHandle_t data_ready = xSemaphoreCreateBinary();
```

Always use `xSemaphoreCreateMutex()` when protecting a shared peripheral or data structure.
Binary semaphore does NOT have priority inheritance and will cause priority inversion.

Summary

Who	Role	Priority	What happens
CEO	Needs the room	High (10)	Blocked outside — waiting
Manager	Does not need room	Medium (5)	Keeps running freely
Intern	Holds the room	Low (1)	Cannot finish — Manager keeps CPU
FreeRTOS	Detects the problem	—	Promotes Intern to priority 10
Intern	Now promoted	Temporary 10	Finishes quickly and exits
CEO	Gets the room	High (10)	Enters and does his work

One Line Summary

"Priority inversion is when a low priority intern accidentally blocks the CEO — because a medium priority manager keeps running and prevents the intern from finishing. FreeRTOS mutexes fix this

automatically by temporarily promoting the intern to CEO level. This exact bug crashed a NASA spacecraft on Mars in 1997."

```
Task HIGH (priority 10) – needs resource X
Task LOW  (priority 1) – holds resource X and cannot release it
Task MED  (priority 5) – running and not blocked
```

What happens:

```
Task HIGH waits for resource X
Task LOW cannot run because Task MED keeps running
Task HIGH effectively runs at LOW priority
This is priority inversion – a bug in many production systems
```

Fix: Use mutexes – they implement priority inheritance automatically in FreeRTOS.

4. Queues – Safe Data Transfer Between Tasks

A queue is the correct way to pass data from one task to another.

Why Not Use a Global Variable?

```
/* WRONG – global variable between tasks */
float global_temperature = 0.0f;

void sensor_task(void *arg) {
    while (1) {
        global_temperature = read_dht22(); /* Write */
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void display_task(void *arg) {
    while (1) {
        show_on_oled(global_temperature); /* Read */
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

This looks fine but has a critical bug called a **race condition**:

Timeline – what can go wrong:

```
sensor_task writes first byte of float (0x41)
SCHEDULER SWITCHES to display_task
display_task reads half-written float: 0x41 + garbage
display_task shows wrong temperature on OLED
SCHEDULER SWITCHES back
sensor_task writes second byte (0xD9)
```

Result: display shows corrupted data.
In production: your product shows 350°C to a customer.

Queues Solve This

A queue is a thread-safe FIFO buffer. Only one task can read or write at a time — atomically.

PC Analogy:

Queue = Email inbox

sensor_task SENDS an email with temperature data
display_task READS from inbox when ready

Only one person reads or writes the inbox at a time.
Data is complete before the reader sees it.
No corruption possible.

Creating and Using a Queue

```
#include "freertos/queue.h"

/* Define the data structure to pass */
typedef struct {
    float temperature;
    float humidity;
} sensor_data_t;

/* Create queue — holds 10 items of type sensor_data_t */
static QueueHandle_t sensor_queue;

void app_main(void)
{
    sensor_queue = xQueueCreate(10, sizeof(sensor_data_t));

    if (sensor_queue == NULL) {
        ESP_LOGE(TAG, "Queue creation failed — out of memory");
        return;
    }

    xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL);
    xTaskCreate(display_task, "display", 4096, NULL, 5, NULL);
}
```

```
/* Producer — sends data to queue */
void sensor_task(void *arg)
{
    sensor_data_t data;

    while (1) {
        data.temperature = read_temperature();
        data.humidity    = read_humidity();
```

```

    /* Send to queue — timeout 0 means don't wait if full */
    if (xQueueSend(sensor_queue, &data, 0) != pdTRUE) {
        ESP_LOGW(TAG, "Queue full — dropped reading");
    }

    vTaskDelay(pdMS_TO_TICKS(2000));
}
}

```

```

/* Consumer — receives data from queue */
void display_task(void *arg)
{
    sensor_data_t data;

    while (1) {
        /* Wait forever until data arrives */
        if (xQueueReceive(sensor_queue, &data, portMAX_DELAY)) {
            show_on_oled(data.temperature, data.humidity);
            ESP_LOGI(TAG, "Temp: %.1f Humi: %.1f",
                data.temperature, data.humidity);
        }
    }
}

```

Queue API Reference

Function	When to use
<code>xQueueCreate(len, size)</code>	Create — do this in <code>app_main</code>
<code>xQueueSend(q, &item, 0)</code>	Send — don't wait if full
<code>xQueueSend(q, &item, portMAX_DELAY)</code>	Send — wait forever if full
<code>xQueueReceive(q, &item, portMAX_DELAY)</code>	Receive — block until data
<code>xQueueReceive(q, &item, 0)</code>	Receive — return immediately if empty
<code>xQueueSendFromISR(q, &item, NULL)</code>	Send from interrupt handler
<code>uxQueueMessagesWaiting(q)</code>	How many items in queue right now
<code>xQueueReset(q)</code>	Empty the queue

5. Semaphores — Protecting Shared Resources

A semaphore is a signalling mechanism between tasks. A mutex is a semaphore with ownership — the task that takes it must be the task that gives it.

Binary Semaphore — Signalling

Use when one task needs to SIGNAL another task that something happened.

```
/* PC Analogy:
   Binary semaphore = a doorbell
   Task A presses doorbell (give)
   Task B waiting at door hears it (take)
   Task B does the work */

static SemaphoreHandle_t data_ready_sem;

void sensor_task(void *arg) {
    while (1) {
        read_sensor();
        xSemaphoreGive(data_ready_sem); /* Ring the doorbell */
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void process_task(void *arg) {
    while (1) {
        /* Wait at the door indefinitely */
        xSemaphoreTake(data_ready_sem, portMAX_DELAY);
        process_data(); /* Doorbell rang – do the work */
    }
}

void app_main(void) {
    data_ready_sem = xSemaphoreCreateBinary();
    xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL);
    xTaskCreate(process_task, "process", 4096, NULL, 5, NULL);
}
```

Mutex — Protecting Shared Resources

Use when multiple tasks need to access the SAME resource (UART, SPI bus, global data structure, I2C bus).

```
/* PC Analogy:
   Mutex = a single key to a shared office bathroom
   Only one person can be inside at a time
   Others wait outside until the key is returned */

static SemaphoreHandle_t i2c_mutex;

void task_a(void *arg) {
    while (1) {
        xSemaphoreTake(i2c_mutex, portMAX_DELAY); /* Take the key */
        /* Only this task uses I2C here */
        i2c_read_sensor();
        xSemaphoreGive(i2c_mutex); /* Return the key */
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```

void task_b(void *arg) {
    while (1) {
        xSemaphoreTake(i2c_mutex, portMAX_DELAY); /* Wait for key */
        /* Now safe to use I2C */
        i2c_write_display();
        xSemaphoreGive(i2c_mutex); /* Return the key */
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void app_main(void) {
    i2c_mutex = xSemaphoreCreateMutex();
    xTaskCreate(task_a, "task_a", 4096, NULL, 5, NULL);
    xTaskCreate(task_b, "task_b", 4096, NULL, 5, NULL);
}

```

Semaphore vs Mutex — When to Use Which

Situation	Use
Task A signals Task B that event happened	Binary semaphore
ISR signals task that interrupt fired	Binary semaphore
Protecting a shared peripheral (I2C, SPI, UART)	Mutex
Protecting a shared data structure	Mutex
Counting available resources (e.g. buffer slots)	Counting semaphore

Golden Rules

- Rule 1: Never call `xSemaphoreTake` inside an ISR
Use `xSemaphoreGiveFromISR` instead
- Rule 2: Always give back what you take
Every Take must have a Give
Forgetting to Give = deadlock
- Rule 3: Keep mutex-protected sections short
Long sections block other tasks unnecessarily
- Rule 4: Never call `vTaskDelay` while holding a mutex
Other tasks waiting for the mutex are starved

6. Complete Production Pattern

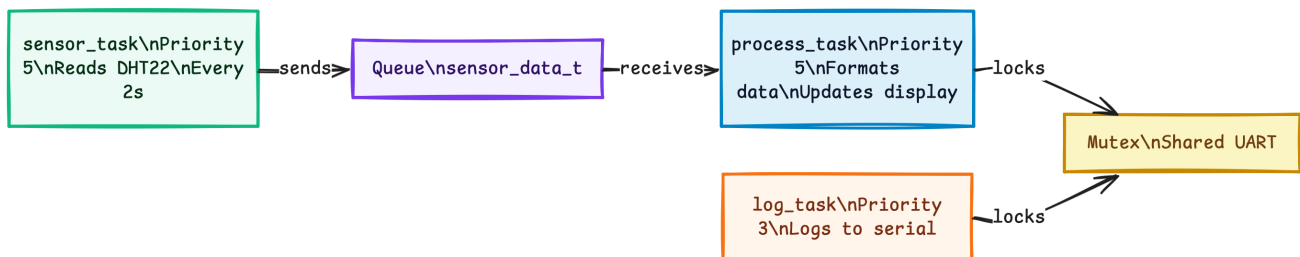
Here is how all three concepts combine in a real product:

```

+-----+           Queue           +-----+
| sensor_task | -----> | process_task |

```


Priority 5	sensor_data_t	Priority 5
Reads DHT22		Formats data
Every 2s		Updates display
+-----+		+-----+
		Mutex (shared UART)
		+-----+
		log_task
		Priority 3
		Logs to serial
		+-----+



```

/* Full pattern - app_main */
void app_main(void)
{
    /* Create communication primitives FIRST */
    sensor_queue = xQueueCreate(10, sizeof(sensor_data_t));
    uart_mutex    = xSemaphoreCreateMutex();

    /* Then create tasks */
    xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL);
    xTaskCreate(process_task, "process", 4096, NULL, 5, NULL);
    xTaskCreate(log_task, "log", 2048, NULL, 3, NULL);

    /* app_main returns - FreeRTOS scheduler takes over */
}

```

7. Debugging FreeRTOS

Check Stack Usage

```

/* Add to any task to see how much stack is left */
void my_task(void *arg) {
    while (1) {
        ESP_LOGI(TAG, "Stack watermark: %u",
                 uxTaskGetStackHighWaterMark(NULL));
        vTaskDelay(pdMS_TO_TICKS(5000));
    }
}

```

```
}
/* If watermark < 256 – increase stack size by 1024 */
```

Enable Stack Overflow Detection

In menuconfig:

```
Component config
--> FreeRTOS
    --> Kernel
        --> Stack overflow detection method
            --> Method 2 (most reliable)
```

When a task overflows its stack — you see:

```
***ERROR*** A stack overflow in task my_task has been detected.
```

List All Running Tasks

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

void print_task_list(void) {
    char buf[512];
    vTaskList(buf);
    ESP_LOGI(TAG, "Task list:\n%s", buf);
}
```

Output:

Name	State	Priority	Stack	Num
sensor_task	R	5	1800	3
display_task	B	5	1600	4
log_task	B	3	1200	5
IDLE	R	0	900	1

Summary Table

Concept	What it is	PC Equivalent	Use when
Task	Independent unit of work	Program / process	Always
Priority	Who runs first	Process priority	Always
vTaskDelay	Sleep without blocking CPU	Thread.sleep()	Always

Concept	What it is	PC Equivalent	Use when
Queue	Thread-safe data pipe	Email inbox	Passing data between tasks
Binary semaphore	One-shot signal	Doorbell	Signalling events
Mutex	Exclusive resource lock	Bathroom key	Shared peripherals
Stack watermark	How much stack left	Stack profiler	Debugging memory
portMAX_DELAY	Wait forever	Blocking call	Consumer tasks
pdMS_TO_TICKS	Convert ms to ticks	Thread.sleep(ms)	All delays